

Using Reinforcement Learning to solve Lunar Lander environment

Valerio Spagnoli 1973484

Matteo Ventali 1985026

June 15, 2025

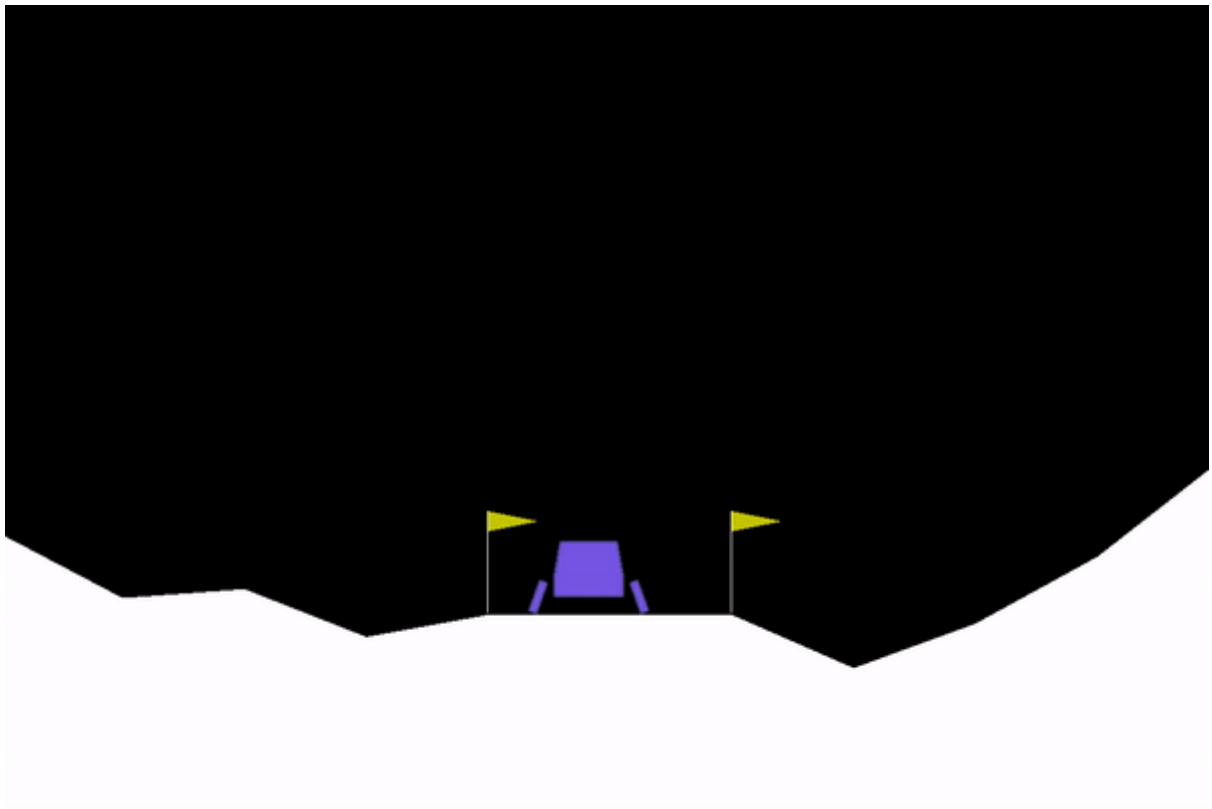
Contents

1	Introduction	2
2	Basics of Q-Learning	3
2.1	Tabular Q-learning	4
2.2	DQN	4
3	Lunar Lander environment	5
3.1	Observation space	5
3.2	Action space	5
3.3	Reward system	5
4	Experiments	6
4.1	Lunar Lander with tabular Q-learning	6
4.1.1	Hyperparameters tuning	7
4.1.2	Training results	8
4.1.3	Running results	10
4.2	Lunar Lander with DQN	11
4.2.1	Hyperparameters tuning	12
4.2.2	Training results	13
4.2.3	Running results	16
4.3	DQN vs Tabular Q-learning	17
5	Conclusion	19
6	Appendix	20

1 Introduction

This project aims to address the Lunar Lander environment by applying a reinforcement learning technique called Q-learning. We will explore two different approaches: tabular Q-learning, which is suitable for environments with small and discrete state spaces, and Deep Q-Networks (DQN), a more advanced method that combines Q-learning with deep neural networks, making it capable of handling larger and continuous state spaces.

The goal is to compare these two techniques in terms of learning performance, stability, and efficiency when applied to the Lunar Lander task, which involves controlling a lander to safely reach the ground between two designated flags under the influence of gravity and physics-based dynamics.



2 Basics of Q-Learning

Q-Learning is a reinforcement learning algorithm used to learn the optimal policy

$$\pi^*(s) : S \rightarrow A$$

for an agent interacting with an environment. It is based on learning a Q-function

$$Q(s, a) : S \times A \rightarrow \mathbb{R}$$

which estimates the expected cumulative (discounted) reward of taking an action a in a state s and then following the optimal policy thereafter.

The critical part of the algorithm is how to maintain and update the Q-function. In particular, at each iteration, the algorithm updates its value with one of the following rules:

- **deterministic rule:**

$$Q^{new}(s, a) \leftarrow r + \gamma \cdot \max_{a' \in A} Q(s', a')$$

- **non deterministic rule:**

$$Q^{new}(s, a) \leftarrow Q(s, a) + \alpha \cdot [r + \gamma \cdot \max_{a' \in A} Q(s', a') - Q(s, a)]$$

In the previous equations appear these terms:

- α : learning rate;
- r : reward received after taking action a in state s ;
- γ : discount factor for future rewards;
- s' and a' : respectively, the next state and the next action.

Algorithm 1 Q-learning Algorithm

- 1: Initialize $Q(s, a)$ arbitrarily for all states s and actions a
 - 2: **for** each episode **do**
 - 3: Initialize state s
 - 4: **while** episode not terminated **do**
 - 5: Choose action a from s using a strategy
 - 6: Execute action a , observe reward r and next state s'
 - 7: Update $Q(s, a)$:
 - 8: **end while**
 - 9: **end for**
-

The next action strategy must balance:

- **exploration:** taking random actions to explore the environment and gather information;

- **exploitation:** selecting actions according to the policy learned so far to maximize rewards.

A commonly used exploration strategy in reinforcement learning is the ε -greedy policy. Under this approach, the agent selects a random action with probability ε , and chooses the action that maximizes the current Q-value estimate with probability $1 - \varepsilon$. This allows the agent to balance exploration and exploitation. To reduce the exploration as training progresses, epsilon decays exponentially over time using a fixed decay rate.

2.1 Tabular Q-learning

In the tabular version of Q-learning, the action-value function $Q(s, a)$ is represented explicitly as a table, where each entry corresponds to a specific state-action pair. During training, this table is updated iteratively using the Q-learning update rule. Tabular Q-learning is effective in environments with small, discrete state and action spaces, where it is feasible to store and update the Q-values for all possible combinations. Its main advantages are simplicity, ease of implementation, and theoretical convergence guarantees. However, this approach does not scale to environments with large or continuous state spaces, as the size of the Q-table grows rapidly and becomes impractical. In such cases, approximating the Q-function with a neural network (DQN) or other approaches becomes necessary.

2.2 DQN

In Deep Q-Learning (DQN), the Q-function is approximated using a neural network instead of a lookup table, making it suitable for environments with large or continuous state spaces.

Unlike traditional supervised learning, where the dataset is fixed and provided in advance, the training data in DQN is generated incrementally through interaction with the environment.

Specifically, during training, the agent builds its dataset in the following way:

- at each time step, the agent observes a transition $(s, a, r, s', done)$ by interacting with the environment;
- this transition is stored in a memory called *replay buffer*;
- periodically, after some interaction steps, a random mini-batch of transitions is sampled from the buffer to train the Q-network.

To apply the Q-learning update rule, the same network is used both to predict the current $Q(s, a)$ and to estimate the target value $Q^{new}(s, a)$. The power of this approach is the ability of the agent to generalize across similar states and learn effective policies in complex environments.

3 Lunar Lander environment

This environment is a classic rocket trajectory optimization problem. The environment is based on a 2D physics simulation powered by Box2D, and the lander must navigate a gravitational field.

3.1 Observation space

The **observation space** consists of 8 continuous variables representing:

- the lander’s x and y positions, ranging from -2.5 to 2.5 ;
- the lander’s velocity components, ranging from -10 to 10 ;
- the lander’s angular velocity, ranging from -10 to 10 ;
- the lander’s angle, ranging from -2π to 2π ;
- two binary flags that represent whether each leg is in contact with the ground or not.

The landing pad is always at coordinates $(0, 0)$. Given the continuity of the observation space, a discretization approach was adopted for the implementation of tabular Q-learning.

3.2 Action space

The environment provides a discrete **action space**, composed by 4 main actions, encoded by 4 integers that range from 0 to 3.

- 0: do nothing;
- 1: fire left orientation engine (pushing right);
- 2: fire main engine (pushing upward);
- 3: fire right orientation engine (pushing left);

3.3 Reward system

The agent receives a reward based on the quality of the landing:

A successful landing near the center earns positive rewards. Crashes or going out of bounds result in large negative rewards. Fuel usage penalizes the agent slightly, encouraging efficiency. Leg contact with the ground provides small bonuses. The episode terminates when the lander comes to rest (landed or crashed), making the problem episodic.

A final reward > 200 is considered a success.

4 Experiments

4.1 Lunar Lander with tabular Q-learning

To apply tabular Q-learning to the Lunar Lander environment, it is necessary to adapt the algorithm to handle the continuous state space provided by the simulator. Since tabular methods require a finite set of discrete states, the continuous observation space—which includes position, velocity, angle, and leg contact information—must be discretized. This typically involves dividing each dimension of the state vector into a fixed number of bins, effectively mapping continuous observations to a limited set of discrete states. This discretization step is crucial, as it enables the use of a Q-table to store and update action-value estimates for each state-action pair. However, it also introduces a trade-off between precision and tractability: finer discretization improves state resolution but increases the size of the Q-table, potentially making learning slower and more memory-intensive.

In our project, the discretization was manually designed to strike a balance between state granularity and computational feasibility. Instead of relying on automatic binning or predefined wrappers, we defined a set of thresholds for each dimension of the observation vector based on empirical intuition and experimentation. This handcrafted approach allowed us to capture the most relevant features of the environment. The following Python function implements this custom discretization:

```
1 def discretize(obs):
2     result = []
3
4     # Interval array
5     x_intervals = [-0.5, 0.5]
6     y_intervals = [-0.1, 0.1, 1.5]
7     vx_intervals = [-7.5, -5, -0.3, -0.1, 0.1, 0.3, 5, 7.5]
8     vy_intervals = [-7.5, -5, -0.3, -0.1, 0.1, 0.3, 5, 7.5]
9     theta_intervals = [-1.25663706, -0.1, 0.1, 1.25663706]
10    omega_intervals = [-7.5, -5, -0.1, 0.1, 5, 7.5]
11
12    result.append(np.digitize(obs[0], x_intervals))
13    result.append(np.digitize(obs[1], y_intervals))
14    result.append(np.digitize(obs[2], vx_intervals))
15    result.append(np.digitize(obs[3], vy_intervals))
16    result.append(np.digitize(obs[4], theta_intervals))
17    result.append(np.digitize(obs[5], omega_intervals))
18
19    # No need to discretize boolean variables
20    result.append(int(obs[6]))
21    result.append(int(obs[7]))
22    return tuple(result)
```

To implement in a efficient way the Q-table we have used the data structure called *defaultdict* as shown in the following snippet of code:

```

1  def tabular_QLearning(self, modality=1): # 0 for random, 1 for eps-greedy policy
2      # - when a state is not already in the dictionary, then it is added with a value of
      ↪ (0,0,0,0). This happens the first time
3      # the agent ends up in that state
4      self.q_table = defaultdict(lambda: np.zeros(self.env.action_space.n))
5
6      . . .
7

```

4.1.1 Hyperparameters tuning

To obtain the optimal hyperparameters, a tuning process was carried out. This involved experimenting with different configurations to evaluate their impact on performance. The plot below illustrates the results of three distinct configurations, highlighting how their performance evolved over time and helping identify the most effective setup.

The configurations have been executed over 15000 episodes with the following parameters:

- Green curve, with $\gamma = 0.9$ and ε -decay = 0.99;
- Blue curve, with $\gamma = 0.9$ and ε -decay = 0.999;
- Red curve, with $\gamma = 0.99$ and ε -decay = 0.9995.

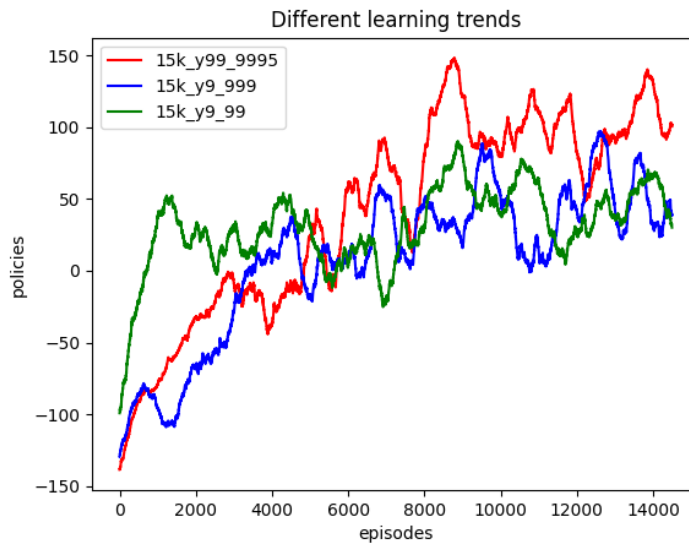


Figure 1: Comparison between three different configuration

The graph shows three different trends based on varying values of gamma and ε -decay. We can see that a fast convergence does not always lead to an optimal policy. In some cases, the agent may quickly settle on suboptimal behaviors due to insufficient exploration or premature exploitation. In fact, although the green line converges almost instantly, the resulting trend stabilizes at relatively lower values, indicating suboptimal performance. By increasing the

value of both gamma and ε -decay, learning becomes slower and more stable, allowing the agent to gradually refine its policy. Only after a sufficient number of episodes does the agent begin to consolidate an optimal policy, demonstrating the importance of careful hyperparameter tuning for long-term performance.

We find further information by seeing the following running results:

γ	ε -decay	avg reward
0.9	0.99	126.14
0.9	0.999	128.03
0.99	0.9995	132.32

Table 1: Average cumulative reward over 1000 episodes

Although the red curve shows the largest gap compared to the others, the overall performance remains similar. This can be attributed to the discretization of the state space.

Thereafter, we select the red curve configuration because the running performance is comparable across all setups, so the choice is based on the training curves.

4.1.2 Training results

The following section presents the results obtained during the training phase rerunning the configuration chosen in the tuning process. These experimental outcomes illustrate how the performance of the learned policy evolves over time, highlighting learning progress and convergence properties. Key metrics such as cumulative reward expressed as a moving average and training stability are used to measure the effectiveness of the tuned hyper-parameters. The graph includes a comparison between a baseline run using a purely random policy (green line) and the training run (red line) using the ε -greedy strategy, showing both the progression of rewards and the variation of ε across episodes (orange line).

The following results have been obtained using following hyper-parameters:

- $\alpha = 0.1$
- $\gamma = 0.99$
- ε decay = 0.9995
- start $\varepsilon = 1.0$, min $\varepsilon = 0.01$

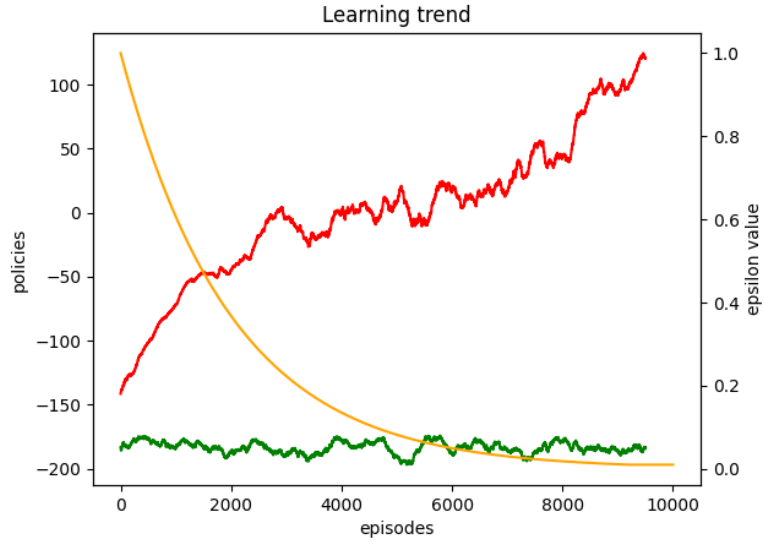


Figure 2: Tabular training process over 10000 episodes

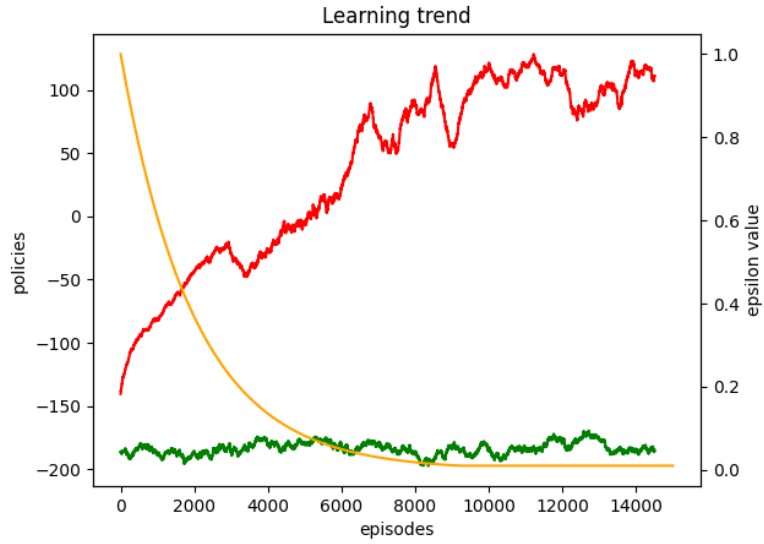


Figure 3: Tabular training process over 15000 episodes

In the first graph, we observe a steadily increasing trend in the cumulative episode reward, indicating that the agent is learning over time. However, the reward curve has not yet reached a plateau, suggesting that the learning process has not fully saturated. To address this limitation, the number of training episodes was extended from 10,000 to 15,000. The resulting trend, shown in the second graph, clearly demonstrates that the agent was able to further consolidate its policy, benefiting from a longer exploitation phase that allowed more stable and consistent improvement. This is supported by the fact that the plateau phase in the second graph is reached when epsilon falls within the range $[0.01, 0.05]$, meaning the agent performs random exploration with a probability between 1% and 5%, and follows the learned policy with a high certainty ranging from 95% to 99%.

4.1.3 Running results

The following section presents the results obtained by running the learned policy after 15,000 training episodes. In this evaluation phase, the agent interacts with the environment for 1,000 episodes using the final policy without further learning. For each episode, the cumulative reward is recorded, and the results are visualized in a scatter plot.

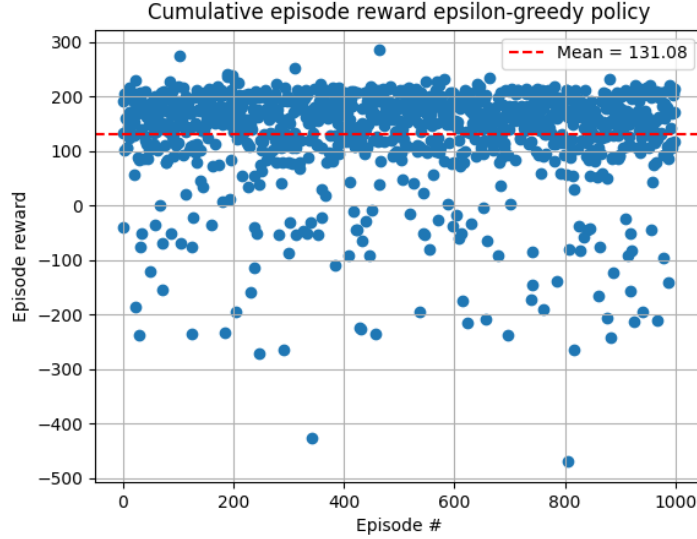


Figure 4: Scatter plot of policy learned by using ϵ -greedy strategy

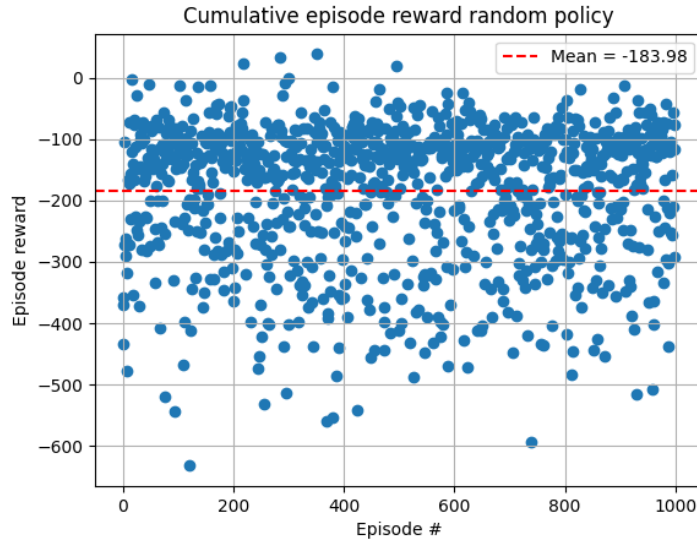


Figure 5: Scatter plot using the random policy

As shown in the first graph, the average reward per episode is approximately 131.08. This suggests that, in most cases, the agent is able to land safely, although not necessarily within the designated landing area marked by the flags (achieving such precision would typically result in a reward greater than 200). Additionally, we observe the presence of several outlier

episodes where the reward drops drastically into negative values. This phenomenon significantly lowers the overall average reward and can be attributed to the discretization of the state space. In the adopted discretization scheme, each feature is divided with higher resolution around zero (i.e., the mean value of each interval is centered closer to 0). However, during the evaluation run, the agent may encounter states that were not visited during training. This mismatch introduces an element of randomness in the action selection, leading to unstable or suboptimal behavior in some cases.

4.2 Lunar Lander with DQN

In our DQN implementation for Lunar Lander, the Q-network is structured as a simple feedforward neural network designed to approximate the action-value function $Q(s, a)$. The network consists of two hidden layers, each equipped with a *ReLU* activation function, and an output layer that produces a Q-value for each possible action. Specifically, the input layer takes in input the continuous state vector from the environment (8 values), while the output layer has one unit per discrete action available in Lunar Lander (4 actions). In our project, the following Python code (using PyTorch library) defines the structure of the network:

```

1  def __init__(self, state_dim, num_actions, device="cpu"):
2      self.device = device
3      self.model = nn.Sequential(
4          nn.Linear(state_dim, 64),
5          nn.ReLU(),
6          nn.Linear(64, 64),
7          nn.ReLU(),
8          nn.Linear(64, num_actions)
9      ).to(self.device)
10     self.loss_fn = nn.MSELoss()
11     self.optimizer = optim.Adam(self.model.parameters(), lr=0.001)

```

As we already said, during training, the agent samples mini-batches of transitions from the replay buffer to update the Q-network. The following Python code demonstrates how these batches are prepared from stored experiences:

```

1  def _prepareBatch(self, batch, q_network: DQN):
2      training_set = [] # Result of preparing the data
3
4      # Preparing the batch
5      for t in batch: # (s, a, r, s', done)
6          # Q(s,a) and Q(s',a) for all action a
7          q_values_s = q_network.predict_qValue(t[0])[0]
8          q_values_ns = q_network.predict_qValue(t[3])[0]
9
10         # Q(s,a) = Q(s,a) + alpha[r + y*max_a'{Q(s',a')}] - Q(s,a)]
11         # Updating only in correspondence of the action
12         action = int(t[1])
13         row = (t[0], q_values_s)
14         if self.update_modality == 0:

```

```

15         row[1][action] = t[2] + self.gamma * np.max(q_values_ns) * (1 - int(t[4])) #det
16     else:
17         row[1][action] = q_values_s[action] + self.alpha * (t[2] + self.gamma *
18             ↪ np.max(q_values_ns) * (1 - int(t[4])) - q_values_s[action]) #nodet
19
19     training_set.append(row)
20
21     return training_set

```

4.2.1 Hyperparameters tuning

Next there are a series of hyperparameters tuning operations. Initially, the focus was on adjusting the ε -decay rate and the discount factor γ . Following this, the tuning process was extended to the replay buffer size to ensure efficient memory usage and learning stability.

The following graph shows the learning trends over several configuration:

- Green curve, with $\gamma = 0.9$ and ε -decay = 0.99;
- Blue curve, with $\gamma = 0.9$ and ε -decay = 0.999;
- Red curve, with $\gamma = 0.99$ and ε -decay = 0.9995.

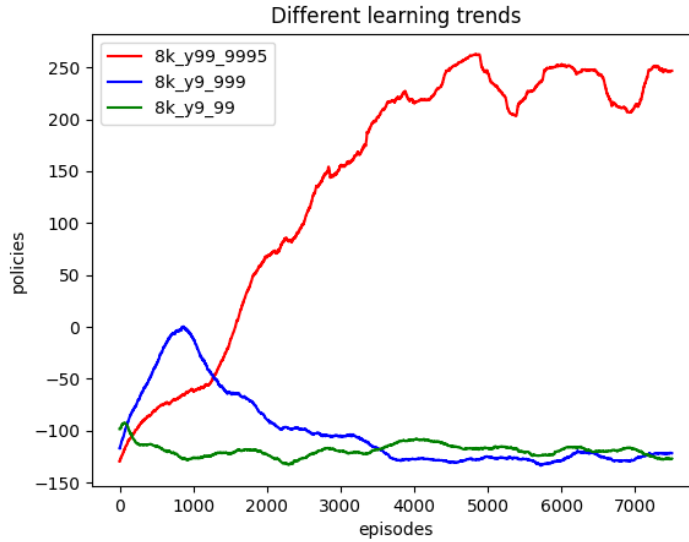


Figure 6: Tuning of γ and ε -decay with non deterministic update rule

As the graph shows, it's evident that the optimal configuration is the one with $\gamma = 0.99$ and ε -decay = 0.9995. The other ones don't give the agent enough time to explore, causing it to focus on a suboptimal policy. Tests were carried out with buffer size fixed at 200,000 and $\alpha = 0.1$.

Subsequently, further tuning was performed on the replay buffer size, while keeping the values of ε -decay and γ fixed at the optimal configuration determined in the previous step.

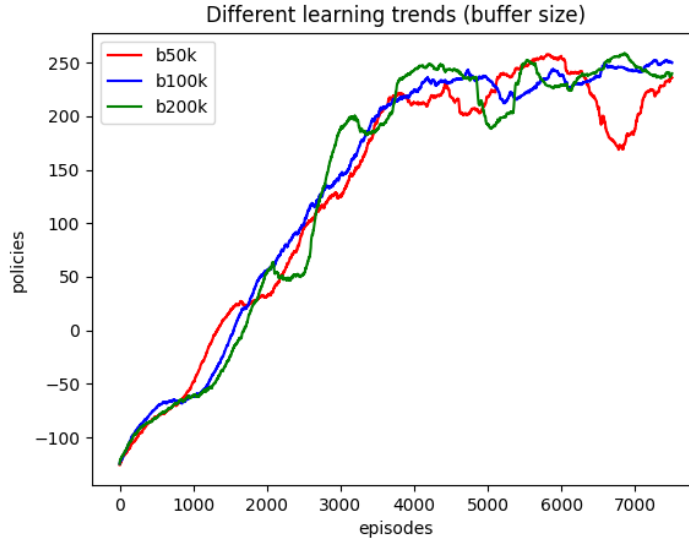


Figure 7: Tuning of the buffer size with non deterministic update rule

At first glance, all three configurations appear to yield similar results, with comparable convergence times and similar value plateaus. Therefore, to properly evaluate their performance, it is necessary to run the learned policy.

buffer size	avg reward
50k	188.14
100k	257.44
200k	269.32

Table 2: Average cumulative reward over 1000 episodes

The performance obtained with a buffer size of 50k is good, but does not reach the level of the policies trained with larger buffer sizes of 100k and 200k. It is also evident that the best configuration is achieved with a buffer size of 200k.

We can attribute this result to the fact that a smaller buffer increases the temporal correlation between experiences, reducing the effectiveness of the replay mechanism.

4.2.2 Training results

During the training phase of the Deep Q-Network (DQN), experiments were conducted over a total of 8000 episodes to evaluate the effectiveness of two different update strategies: one based on a *deterministic rule* and the other on a *non-deterministic rule*. The goal of this comparison was to analyze the impact of the two update mechanisms on the learning process and the final performance of the agent. This setup was used to investigate how the network behaves under softer, more gradual updates (using non-deterministic formula with $\alpha = 0.1$) versus more aggressive ones. In both cases, the training parameters and simulation environment were kept constant to ensure a fair and consistent comparison.

The hyperparameters were selected based on the outcomes of the tuning process. In

particular, $\gamma = 0.99$, ε -decay = 0.9995, and the replay buffer size to 200,000 records and sample minibatch size = 64.

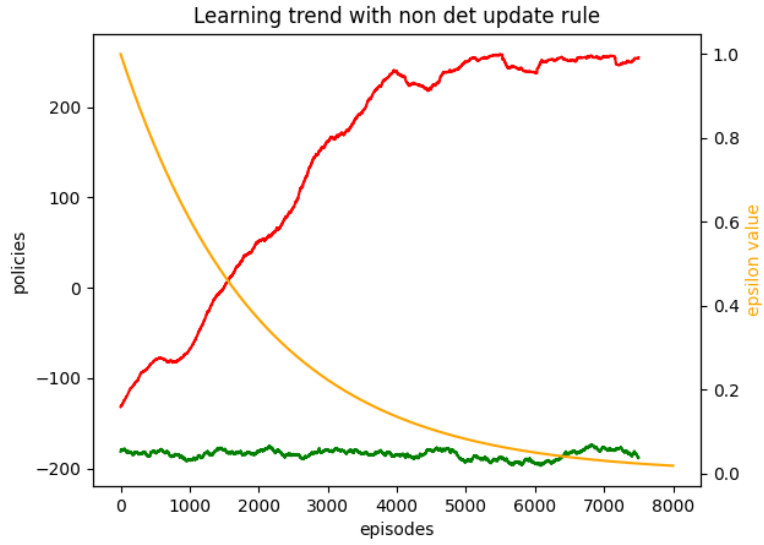


Figure 8: Tabular training process over 8000 episodes with non-deterministic rule

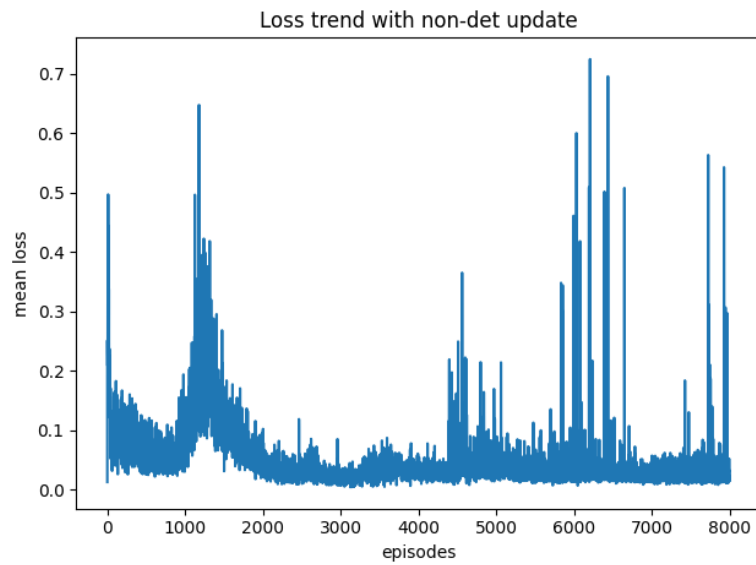


Figure 9: Average loss per episode using non-deterministic rule

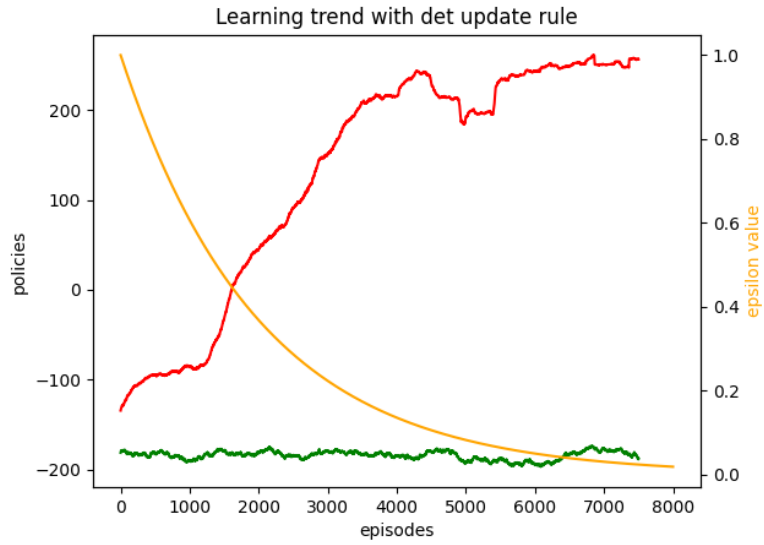


Figure 10: Tabular training process over 8000 episodes with deterministic rule

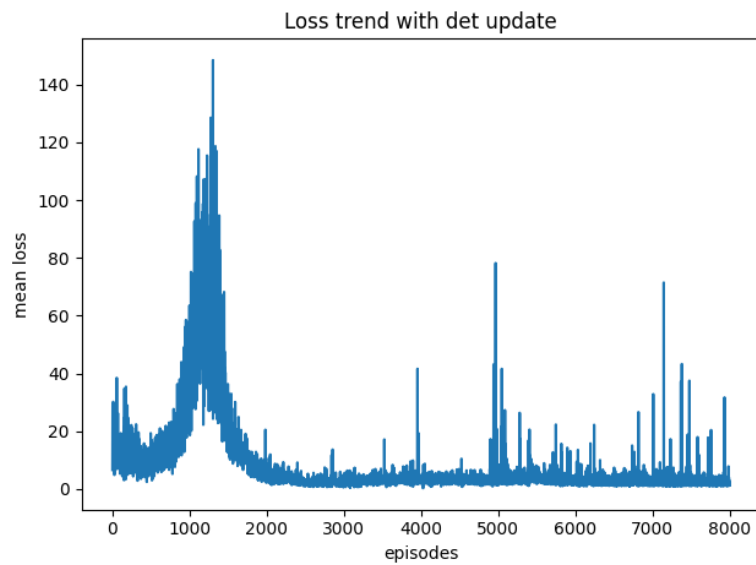


Figure 11: Average loss per episode using deterministic rule

The two graphs show how a Deep Q-Network learns using two different rules: one deterministic and one non-deterministic. Both versions are able to learn well and reach a stable reward of about 250, with similar speed and performance. However, the loss curves are very different. In the non-deterministic approach, the usage of α leads to a soft update to the target, making loss trends values ranging in different intervals. Still, both versions show an initial peak in the loss followed by stabilization, which shows that the network is learning correctly before reaching a steady state.

Next there is a comparison between the two learning trends.

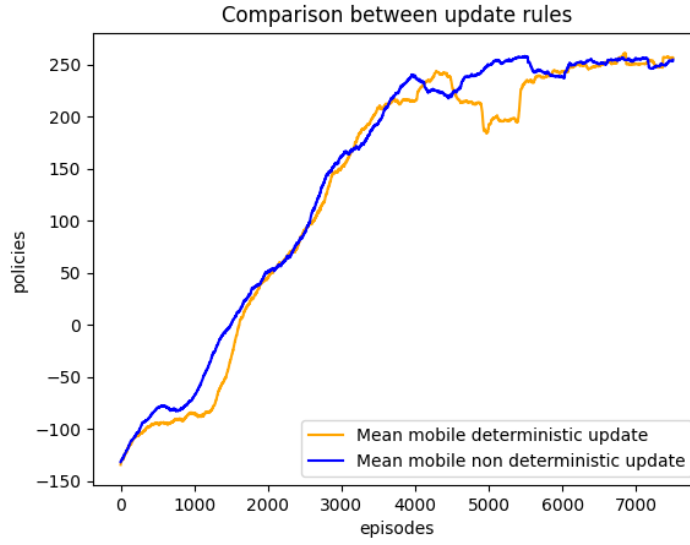


Figure 12: Average loss per episode using deterministic rule

Based on the graph, it is evident that the learning curves of both the deterministic and non-deterministic update rules exhibit very similar trends over the course of the episodes. This similarity suggests a high degree of interchangeability between the two approaches, as both lead to comparable performance in terms of policy improvement.

4.2.3 Running results

To evaluate the performance of the learned policies, we measured the average reward obtained over 1000 episodes for each update rule. The results are presented in the form of a scatter plot, allowing a direct visual comparison between the deterministic and non-deterministic approaches.

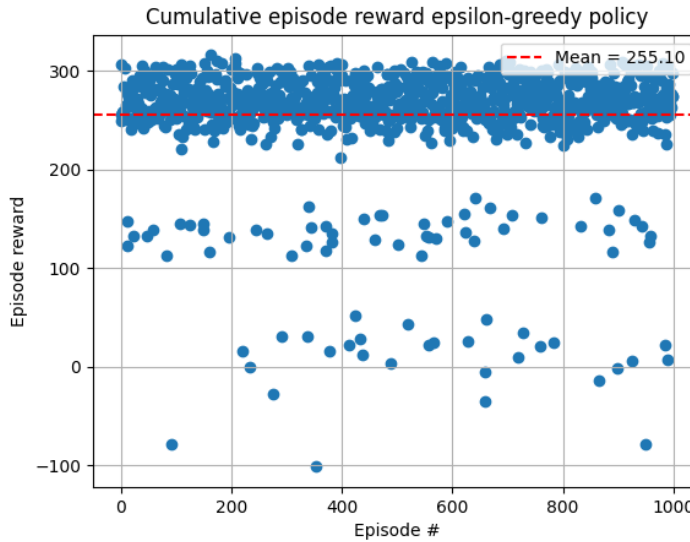


Figure 13: Scatter plot using policy learned with non-deterministic update rule

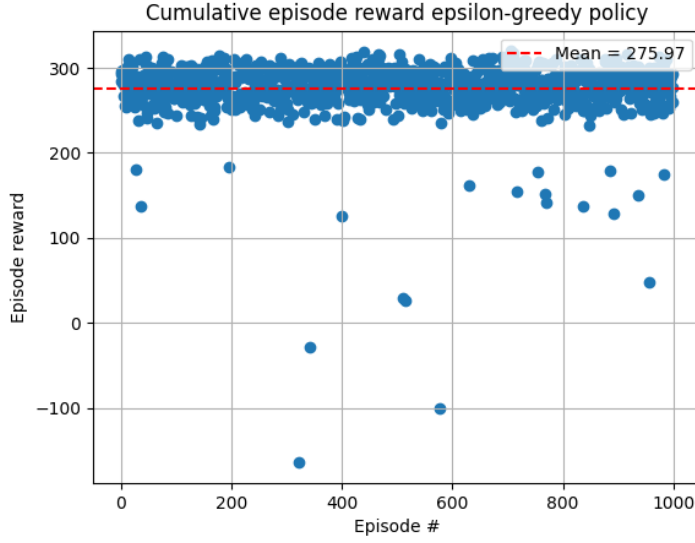


Figure 14: Scatter plot using policy learned with deterministic update rule

In both cases, the policies demonstrate excellent performance, achieving high average rewards over the 1000 evaluation episodes. However, in the deterministic update case, we observe a slightly higher average reward overall, along with a greater concentration of values in the reward range between 250 and 300.

4.3 DQN vs Tabular Q-learning

In this section we present a comparison between the tabular and DQN approaches, evaluating the policies over 1000 episodes and reporting some results.

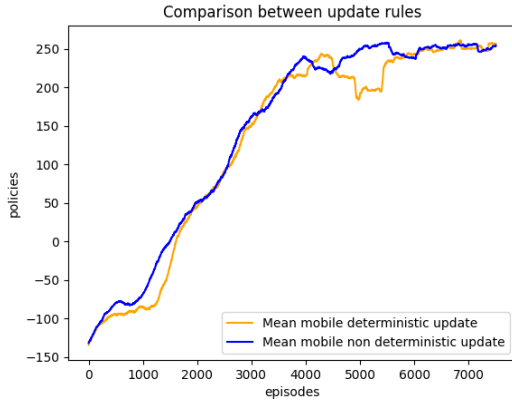


Figure 15: Learning trends using DQN approach

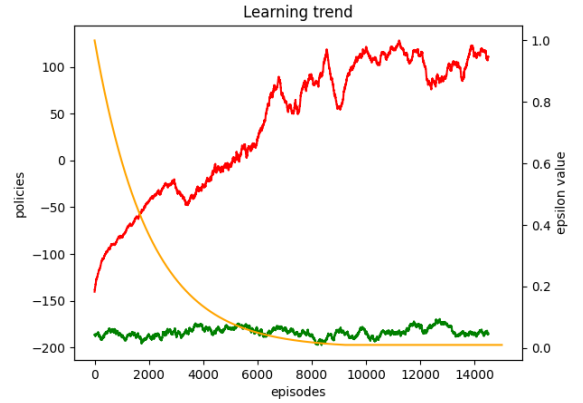


Figure 16: Learning trend using tabular approach

Agent	avg reward	avg steps/episode	# success (≥ 100)	# success (≥ 200)
Tabular	131.08	684.29	822	154
DQN det	275.60	217.9	995	984
DQN non-det	257.16	268.7	971	937

While the DQN approach demonstrates superior overall performance and faster convergence, it is important to note that the results of the tabular method are strongly influenced by the choice of discretization strategy. A well-designed discretization can significantly enhance the performance of tabular Q-learning, as it determines how effectively the continuous state space is represented.

Despite its lower overall reward trend, the average reward achieved by the tabular agent still indicates that the lander successfully performs safe landings in the majority of episodes. This is a very promising result, especially considering the simplicity of the tabular approach. It shows that, with careful design, even classical reinforcement learning methods can yield solid and reliable policies in challenging environments.

5 Conclusion

The goal of this experiment was to solve the Lunar Lander environment using two different reinforcement learning approaches: tabular Q-learning and Deep Q-Networks. The results demonstrate that both methods are capable of learning effective policies, achieving successful landings in the majority of episodes. However, the DQN approach yields superior performance overall, thanks to its ability to generalize optimal actions to previously unseen states. This generalization capability allows the neural network to make more informed decisions in complex and continuous state spaces, leading to more consistent and higher rewards compared to the tabular method.

6 Appendix

We are going to show the implementation of the learning algorithm in both approaches in order to highlight the differences in updating the Q-function $Q(s, a)$, as explained in the previous sections.

```
1  def DQN_Learning(self, modality=1, update_modality=1):
2      self.eps = 1.0
3      self.update_modality = update_modality
4
5      q_network = DQN(8, self.env.action_space.n)
6
7      memory = ReplayBuffer(self.memory_capacity)
8
9      s, _ = self.env.reset()
10
11     total_rewards = []
12     eps_per_episode = []
13     episodes_loss = []
14
15     for n_episode in range(self.max_episodes):
16         episode_reward = 0
17         episode_loss = []
18         done = False
19         n_steps = 0
20         while not done:
21             a = self._next_action(modality, s, q_network)
22
23             ns, reward, terminated, truncated, _ = self.env.step(a)
24             n_steps+=1
25             episode_reward += reward
26             done = terminated or truncated
27
28             memory.add(s, a, reward, ns, done)
29
30             if ( len(memory) > self.batch_dimension and n_steps \% self.update_every == 0
31                 ↪ ):
32                 batch = memory.sample(self.batch_dimension)
33                 training_set = self._prepareBatch(batch, q_network)
34                 loss = q_network.train(training_set)
35                 episode_loss.append(loss)
36
37             if not done:
38                 s = ns
39
40         episodes_loss.append(np.mean(episode_loss))
41         eps_per_episode.append(self.eps)
42         self._epsilon_update()
43         s, _ = self.env.reset()
44         total_rewards.append(episode_reward)
```

```

1  def tabular_QLearning(self, modality=1):
2
3      self.q_table = defaultdict(lambda: np.zeros(self.env.action_space.n))
4      self.eps = 1.0
5
6      s, _ = self.env.reset()
7      s = discretize(s)
8
9      total_rewards = []
10     eps_per_episode = []
11
12     for n_episode in range(self.max_episodes):
13
14         terminated = truncated = False
15         episode_reward = 0
16
17         while not (terminated or truncated):
18             a = self._next_action(s, modality)
19
20             ns, reward, terminated, truncated, _ = self.env.step(a)
21             episode_reward += reward
22             ns = discretize(ns)
23
24             self.q_table[s][a] += self.alpha * (reward + self.gamma *
25 ↪      max(self.q_table[ns]) - self.q_table[s][a])
26
27             if not (terminated or truncated):
28                 s = ns
29
30         eps_per_episode.append(self.eps)
31         self._epsilon_update()
32         s, _ = self.env.reset()
33         s = discretize(s)
34         total_rewards.append(episode_reward)

```